

RAG Document Q&A Test Document

This is a test document for validating the RAG ingestion pipeline. It contains two pages of text about artificial intelligence and machine learning. The purpose is to verify that PDF parsing, chunking, embedding, and storage all work correctly.

Artificial intelligence (AI) is a broad field of computer science focused on building systems capable of performing tasks that typically require human intelligence. These tasks include visual perception, speech recognition, decision-making, and language translation. AI systems can be categorized into narrow AI, which is designed for specific tasks, and general AI, which would have broad cognitive abilities comparable to humans.

Machine learning is a subset of AI that enables systems to learn and improve from experience without being explicitly programmed. It focuses on developing algorithms that can access data and use it to learn for themselves. The process begins with observations or data, such as examples, direct experience, or instruction, to look for patterns in data and make better decisions in the future.

Vector Databases and Embeddings

Vector databases are specialized database systems designed to store, index, and query high-dimensional vector data efficiently. They are essential components in modern AI applications, particularly in retrieval-augmented generation (RAG) systems.

Embeddings are numerical representations of text, images, or other data types in a continuous vector space. Text embeddings capture semantic meaning, so that similar concepts are represented by vectors that are close together in the embedding space. Popular embedding models include OpenAI's `text-embedding-ada-002` and the open-source `all-MiniLM-L6-v2` from Sentence Transformers.

ChromaDB is a lightweight, open-source vector database that supports persistent storage and similarity search. It is commonly used in prototyping and small-scale RAG applications. ChromaDB stores documents alongside their vector embeddings and metadata, enabling filtered similarity searches.

The retrieval-augmented generation pattern works by first converting a user query into an embedding, then finding the most similar document chunks in the vector database, and finally passing those chunks as context to a large language model to generate a grounded answer. This approach reduces hallucination by anchoring the model's response in actual source material.